

Chapter 34: Heaps & Priority Queues

Personal Notes

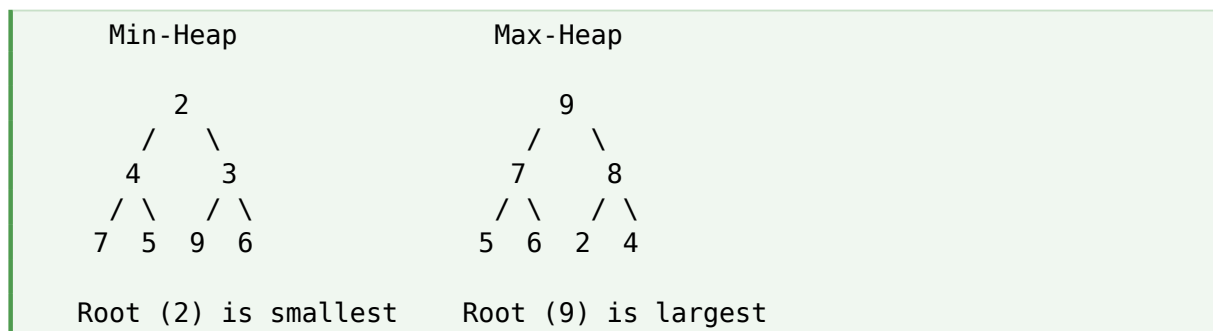
Sometimes you don't want a full sort — you just want the SMALLEST or LARGEST element right now. Or the top 10 fastest runners. Or the closest 5 restaurants. These are PRIORITY problems, and the perfect tool for them is a HEAP.

A heap gives you $O(\log n)$ insert and $O(\log n)$ removal of the min (or max) — far faster than sorting a list every time. In Dijkstra's algorithm (Ch 33), the priority queue was the engine that made it work. This chapter unpacks how heaps actually function under the hood, and shows the family of problems they solve.

1. What is a Heap?

A HEAP is a complete binary tree that satisfies the HEAP PROPERTY: every parent obeys a specific order relative to its children. Two variants:

- Min-Heap — every parent is SMALLER than its children. The smallest value is always at the root.
- Max-Heap — every parent is LARGER than its children. The largest value is always at the root.



Not sorted — just ordered locally: A heap ISN'T fully sorted. In-order traversal doesn't produce a sorted list. What a heap guarantees is that the ROOT is always the min (or max) — and that's all you need for priority problems. This local ordering is why insert and remove are only $O(\log n)$.

2. Why Not Just Sort?

Compare three approaches to "find the smallest element repeatedly":

Approach	Cost	Trade-off
Unsorted array	$O(n)$ scan every time	Slow for repeated queries
Fully sorted array	$O(n \log n)$ initial sort	Insert becomes $O(n)$ — shift elements

Approach	Cost	Trade-off
Heap (Priority Queue)	$O(\log n)$ both insert & remove	The sweet spot for dynamic data

When you have a stream of data and need repeated min/max queries, heaps win — every time.

3. How a Heap Stores Data (Array-Backed)

Even though a heap is a binary tree conceptually, it's usually stored in a simple ARRAY. Because a heap is a COMPLETE binary tree (filled level by level, left to right), parent-child relationships have a clean formula.

Tree view:	Array view:
<pre> 10 / \ 15 20 / \ / \ 30 40 50 25 </pre>	<pre> Index: 0 1 2 3 4 5 6 Value: 10 15 20 30 40 50 25 </pre>
For index i :	
parent = $(i - 1) / 2$	
left = $2 * i + 1$	
right = $2 * i + 2$	

- Parent of index $i \rightarrow (i - 1) / 2$
- Left child of index $i \rightarrow 2 * i + 1$
- Right child of index $i \rightarrow 2 * i + 2$

Why this matters: Because the heap is complete, no space is wasted — just a plain array with no pointers. All those simple index formulas above let you navigate the tree without any references. This makes heaps memory-efficient AND cache-friendly.

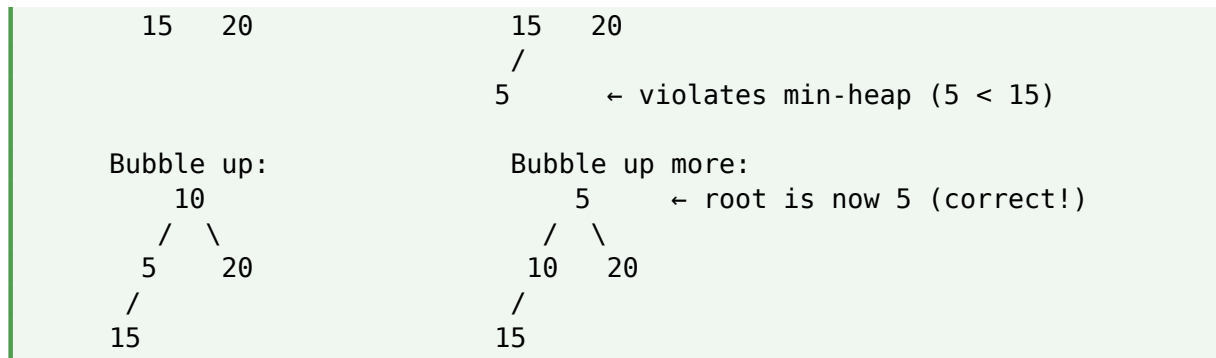
4. Heap Operations — Insert (Bubble Up)

To insert a new value:

1. Add the new value at the END of the array (rightmost, deepest position)
2. Compare with its PARENT — if it violates the heap property, SWAP
3. Keep swapping upward until the property holds or you reach the root

Example: Insert 5 into Min-Heap

Before:	After adding 5 at the end:
<pre> 10 / \ </pre>	<pre> 10 / \ </pre>



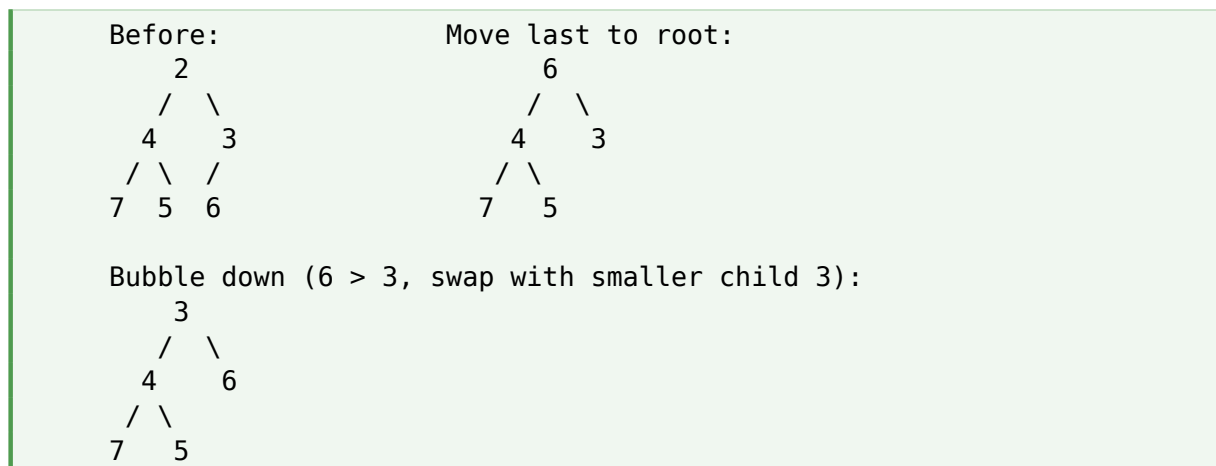
Height of a complete tree is $O(\log n)$, so bubble-up takes $O(\log n)$ time.

5. Heap Operations — Remove Min (Bubble Down)

To remove the smallest element (the root):

4. Save the root value (this is your answer)
5. Move the LAST element (rightmost leaf) to the root
6. Compare the new root with its SMALLER child — if it's bigger, SWAP
7. Keep swapping downward until the property holds

Example: Remove Min from Min-Heap



Also $O(\log n)$ — you walk down at most the height of the tree.

6. Java's PriorityQueue — Ready to Use

Java gives you a battle-tested heap: `PriorityQueue<T>`. It's a MIN-HEAP by default, holds any `Comparable` type, and gives $O(\log n)$ insert and remove.

Basic Usage — Min-Heap

```
import java.util.PriorityQueue;
```

```

PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(30);
pq.offer(10);
pq.offer(20);

pq.peek();    // 10 (min, no removal)
pq.poll();    // 10 (min, removes it)
pq.poll();    // 20
pq.poll();    // 30
pq.size();    // 0

```

Max-Heap

```

// Pass a reverse comparator
PriorityQueue<Integer> maxHeap = new
PriorityQueue<>(Comparator.reverseOrder());
maxHeap.offer(30);
maxHeap.offer(10);
maxHeap.offer(20);

maxHeap.poll();    // 30 (largest first)

```

Custom Objects

```

class Task {
    String name;
    int priority;
    Task(String n, int p) { name = n; priority = p; }
}

// Sort by priority – smallest first
PriorityQueue<Task> tasks = new PriorityQueue<>(
    Comparator.comparingInt(t -> t.priority)
);

tasks.offer(new Task("write email", 3));
tasks.offer(new Task("fix bug", 1));
tasks.offer(new Task("review PR", 2));

tasks.poll();    // "fix bug" (priority 1)
tasks.poll();    // "review PR"
tasks.poll();    // "write email"

```

Common Methods

Method	What it does
offer(x) / add(x)	Insert x — $O(\log n)$
poll()	Remove and return min — $O(\log n)$

Method	What it does
peek()	Look at min without removing — $O(1)$
size()	Number of elements — $O(1)$
isEmpty()	True if empty — $O(1)$
contains(x)	Check if x exists — $O(n)$ (slow)
remove(x)	Remove specific x — $O(n)$ (slow)

PriorityQueue quirks: Iterating a PriorityQueue does NOT visit elements in sorted order — only poll() does. And contains(x) / remove(x) are $O(n)$, not $O(\log n)$. For efficient arbitrary removal you need a TreeSet or a hand-rolled indexed heap. For standard interview problems, PriorityQueue's core operations (offer, poll, peek) are what you'll use 95% of the time.

7. Building a Heap in Java from Scratch

Great for understanding what PriorityQueue does under the hood — and for interviews that ask you to implement one. Here's a clean min-heap.

```
class MinHeap {
    private int[] heap;
    private int size;

    public MinHeap(int capacity) {
        heap = new int[capacity];
        size = 0;
    }

    public void offer(int val) {
        heap[size++] = val;
        bubbleUp(size - 1);
    }

    public int poll() {
        if (size == 0) throw new RuntimeException("Empty heap");
        int min = heap[0];
        heap[0] = heap[--size];
        bubbleDown(0);
        return min;
    }

    public int peek() { return heap[0]; }
    public int size() { return size; }
}
```

```

private void bubbleUp(int i) {
    while (i > 0) {
        int parent = (i - 1) / 2;
        if (heap[i] < heap[parent]) {
            swap(i, parent);
            i = parent;
        } else break;
    }
}

private void bubbleDown(int i) {
    while (true) {
        int left = 2 * i + 1, right = 2 * i + 2, smallest = i;
        if (left < size && heap[left] < heap[smallest])    smallest =
left;
        if (right < size && heap[right] < heap[smallest]) smallest =
right;
        if (smallest == i) break;
        swap(i, smallest);
        i = smallest;
    }
}

private void swap(int a, int b) {
    int tmp = heap[a];
    heap[a] = heap[b];
    heap[b] = tmp;
}
}

```

8. Classic Problem 1 — Kth Largest Element

Given an array of integers, find the Kth largest. Naive approach sorts ($O(n \log n)$). Heap approach uses a MIN-HEAP of size K — beats sort for small K.

The Trick

- Keep a min-heap of size K containing the K largest values seen so far
- For each new number, add it to the heap
- If the heap exceeds size K, remove the minimum
- At the end, the min of the heap IS the Kth largest overall

Code

```

public static int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> pq = new PriorityQueue<>();
    for (int n : nums) {
        pq.offer(n);
    }
}

```

```

        if (pq.size() > k) pq.poll();
    }
    return pq.peek();
}

// Time: O(n log k) – much better than sort for k << n
// Space: O(k)

```

Dry Run for [3, 2, 1, 5, 6, 4], k = 2

```

n=3: heap=[3]
n=2: heap=[2, 3]      size=2, ok
n=1: heap=[1, 3, 2]   size=3 > 2, poll min (1) → heap=[2, 3]
n=5: heap=[2, 3, 5]   poll min (2) → heap=[3, 5]
n=6: heap=[3, 5, 6]   poll min (3) → heap=[5, 6]
n=4: heap=[4, 6, 5]   poll min (4) → heap=[5, 6]

Result: peek() = 5      ← 2nd largest of the array

```

9. Classic Problem 2 — Top K Frequent Elements

Given an array and integer K, return the K most frequent elements. Combine HashMap (frequency count from Ch 20) with a min-heap (this chapter).

```

public static List<Integer> topKFrequent(int[] nums, int k) {
    // Step 1: count frequencies
    Map<Integer, Integer> freq = new HashMap<>();
    for (int n : nums) freq.put(n, freq.getOrDefault(n, 0) + 1);

    // Step 2: min-heap of size k, ordered by frequency
    PriorityQueue<int[]> pq = new PriorityQueue<>(
        (a, b) -> a[1] - b[1]    // smaller frequency first
    );

    for (var e : freq.entrySet()) {
        pq.offer(new int[]{e.getKey(), e.getValue()});
        if (pq.size() > k) pq.poll();
    }

    // Step 3: extract results
    List<Integer> result = new ArrayList<>();
    while (!pq.isEmpty()) result.add(pq.poll()[0]);
    return result;
}

// Time: O(n log k)

```

10. Classic Problem 3 — Merge K Sorted Lists

Given K sorted linked lists (or arrays), merge them into one sorted list. A perfect heap problem — always pull the smallest head across all K lists.

```
// Node class assumed
public static ListNode mergeKLists(ListNode[] lists) {
    PriorityQueue<ListNode> pq = new PriorityQueue<>((a, b) -> a.val - b.val);

    // Add the head of each list to the heap
    for (ListNode head : lists) {
        if (head != null) pq.offer(head);
    }

    ListNode dummy = new ListNode(0), tail = dummy;
    while (!pq.isEmpty()) {
        ListNode smallest = pq.poll();
        tail.next = smallest;
        tail = smallest;
        if (smallest.next != null) pq.offer(smallest.next);
    }

    return dummy.next;
}

// Time: O(N log K) — N total elements, K lists
```

The heap size is only K: Not N. At any moment, we have exactly one element per list in the heap (or fewer as lists get exhausted). That's why the log K factor is small even for huge total N — this is dramatically better than merging pair-by-pair.

11. Classic Problem 4 — Median from Data Stream

You're given numbers one at a time. After each addition, report the median so far. Classic "two heaps" pattern.

The Two-Heaps Trick

- MAX-heap for the smaller half of the numbers
- MIN-heap for the larger half
- Keep sizes balanced (differ by at most 1)
- Median is either the top of one heap, or the average of the two tops

Numbers so far: 1, 2, 3, 4, 5

Max-heap (smaller half): 3, 2, 1 top: 3

Min-heap (larger half): 4, 5 top: 4

Median = 3 (max-heap top since it has more elements)

After adding 6:

Max-heap: 3, 2, 1

top: 3

Min-heap: 4, 5, 6

top: 4

Median = $(3 + 4) / 2 = 3.5$

```
class MedianFinder {
    PriorityQueue<Integer> low = new
PriorityQueue<>(Comparator.reverseOrder()); // max-heap
    PriorityQueue<Integer> high = new PriorityQueue<>();
// min-heap

    public void addNum(int num) {
        low.offer(num);
        high.offer(low.poll()); // move max of low
to high

        // Balance sizes: low >= high
        if (high.size() > low.size()) low.offer(high.poll());
    }

    public double findMedian() {
        if (low.size() > high.size()) return low.peek();
        return (low.peek() + high.peek()) / 2.0;
    }
}
```

12. Heap Sort

Since a heap gives us min (or max) in $O(\log n)$, we can sort by repeatedly extracting: $O(n \log n)$ overall — the same as merge sort, but with in-place potential.

```
public static int[] heapSort(int[] arr) {
    PriorityQueue<Integer> pq = new PriorityQueue<>();
    for (int n : arr) pq.offer(n);

    int[] sorted = new int[arr.length];
    for (int i = 0; i < arr.length; i++) sorted[i] = pq.poll();
    return sorted;
}
```

Real heap sort is done in-place on the input array using clever heapify, but this version is easier to read and works fine for most interview settings.

13. Complexity Summary

Operation	Time	Note
Build heap from n items	$O(n)$ (heapify)	$O(n \log n)$ if inserting one-by-one
Insert (offer)	$O(\log n)$	-
Remove min (poll)	$O(\log n)$	-
Peek	$O(1)$	-
Contains(x)	$O(n)$	Linear scan
Kth largest	$O(n \log k)$	-
Top K frequent	$O(n \log k)$	-
Merge K sorted lists	$O(N \log k)$	N = total elements
Heap sort	$O(n \log n)$	-

14. When to Use a Heap

If you need to...	Use
Kth largest / Kth smallest in an array	Min-heap or max-heap of size K
Top K frequent elements	Frequency map + heap of size K
Merge K sorted lists or arrays	Min-heap of heads (or first elements)
Running median from a data stream	Two heaps (max + min)
Task scheduling by priority	PriorityQueue<Task> keyed by priority
Dijkstra's / Prim's algorithm	Min-heap of (distance, node)
A* pathfinding heuristic	Min-heap of (estimated cost, state)
Meeting rooms / event scheduling	Heap of end times (or start times)

15. Common Mistakes

Mistake 1: Assuming iteration is sorted

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(3); pq.offer(1); pq.offer(2);
```

```
for (int n : pq) System.out.print(n + " ");    // x may print "1 3 2"
// Only poll() gives sorted output!
```

Mistake 2: Using natural order for max-heap

```
// Default is MIN-heap, not max
PriorityQueue<Integer> pq = new PriorityQueue<>();

// For max-heap:
PriorityQueue<Integer> pq = new
PriorityQueue<>(Comparator.reverseOrder());
```

Mistake 3: Overflow in subtraction comparators

```
// Can overflow for very large/small ints
PriorityQueue<Integer> pq = new PriorityQueue<>((a, b) -> a - b);

// Safe:
PriorityQueue<Integer> pq = new PriorityQueue<>(Integer::compare);
```

Mistake 4: Expecting $O(\log n)$ for contains or remove

These are $O(n)$ in a standard heap. If you need frequent arbitrary lookups, use a `TreeSet` or maintain a separate `HashMap`.

Mistake 5: Modifying elements after inserting

If you insert an object then change its priority field, the heap's internal ordering is broken. Always insert immutable snapshots, or remove and re-insert when priority changes.

Mistake 6: Wrong heap direction for the problem

For "Kth LARGEST," use a MIN-heap of size K (the min of the heap is the Kth largest overall). Mixing this up is the classic top-K mistake.

16. Best Practices

- Use Java's `PriorityQueue` for 95% of interview problems
- For max-heap, always pass `Comparator.reverseOrder()` explicitly — never rely on negation tricks
- Prefer `Integer.compare` over subtraction to avoid overflow
- For custom objects, use `Comparator.comparingInt` or `comparingLong` for cleaner code
- Keep the heap SIZE-K for top-K problems — don't collect everything and sort
- Use two heaps (max + min) for median-of-stream problems
- Never rely on iteration order — only `poll()` gives sorted output
- For custom min/max lookups with efficient removal, consider `TreeMap/TreeSet` instead

17. Quick Reference

Concept	Meaning
Heap	Complete binary tree with the heap property
Min-Heap	Parent $\leq$ children; root = min
Max-Heap	Parent $\geq$ children; root = max
Bubble up	Move new value upward after insert
Bubble down	Move new root downward after remove
Array-backed	Heap stored in a plain array using index math
PriorityQueue	Java's built-in heap (min-heap by default)
offer(x)	Insert — $O(\log n)$
poll()	Remove and return min — $O(\log n)$
peek()	Look at min — $O(1)$
Comparator.reverseOrder()	Turn min-heap into max-heap
Two-heap trick	Max-heap + Min-heap for running median

18. Practice Problems

Try these in order — top-K and two-heap patterns dominate real interviews.

Warm-ups

8. Use PriorityQueue to sort an array in ascending order.
9. Use PriorityQueue with reverseOrder to sort in descending order.
10. Build a min-heap from scratch (bubble up + bubble down). Test offer and poll.
11. Find the smallest and largest elements in a list using a single heap operation.

Top-K Family

12. Kth Largest Element in an Array.
13. Kth Smallest Element in an Array.
14. Top K Frequent Elements.
15. Top K Frequent Words.

16. K Closest Points to Origin (given points and value k).
17. Sort Characters by Frequency.

Merge Family

18. Merge two sorted lists using a heap of head pointers.
19. Merge K Sorted Lists.
20. Merge K Sorted Arrays.
21. Smallest Range Covering Elements from K Lists.

Two-Heap

22. Find Median from Data Stream.
23. Sliding Window Median.

Priority-based

24. Task Scheduler — with cooldowns between same-priority tasks.
25. Meeting Rooms II — minimum rooms needed given start/end times.
26. Reorganize String — arrange characters so no two adjacent are equal.
27. Furthest Building You Can Reach — bricks vs ladders, min-heap of gaps.

19. Final Takeaway

Heaps are a small topic but they punch far above their weight in interviews. Once you internalize the "heap of size K" pattern for top-K problems, and the "two heaps" pattern for running median, you can solve a huge slice of medium-hard interview questions.

In Java, you almost never build a heap by hand — `PriorityQueue` is production-ready and handles both min and max cases through the comparator. The skill is knowing WHEN to reach for it, and structuring your algorithm around the $O(\log n)$ insert/remove sweet spot.

Key Takeaway: Heap = $O(\log n)$ insert and remove, $O(1)$ peek. Use Java's `PriorityQueue`. Min-heap by default; pass `Comparator.reverseOrder()` for max-heap. Master three patterns: (1) top-K with size-K heap, (2) merge-K with heap of heads, (3) running median with two heaps. These three patterns cover almost every heap interview problem you'll ever see.

20. What's Next?

With heaps done, the DSA toolkit is almost complete. Remaining topics:

- Tries — prefix trees for autocomplete, word search, spellcheck
- Union-Find (DSU) — fast connectivity and grouping
- Sliding Window — efficient subarray and substring patterns

- Two-Pointer patterns in depth
- Bit Manipulation — subset generation, XOR tricks
- Advanced Graph Algorithms — Minimum Spanning Tree (Kruskal, Prim), Bellman-Ford
- Segment Tree / Fenwick Tree — range queries and point updates
- String algorithms — KMP, Rabin-Karp for pattern matching